ISLAMIC UNIVERSITY OF TECHNOLOGY

Board Bazar, Gazipur, Dhaka

Machine Learning Assignment 1 - CSE 4553

STUDENT ID - 210042112

Fifth Semester
Department of Computer Science and Engineering
BSc. in Software Engineering

March 15, 2025

Abstract

The BdSLW60 dataset is an innovative collection designed specifically for advancing research in **Bangladeshi Sign Language (BdSL)** recognition using machine learning. Compiled meticulously, this dataset features **9307 distinct sign instances**, covering **60 common BdSL words**, offering a comprehensive range from a **minimum of 9 frames to a maximum of 164 frames per gesture**, averaging around 44 frames per instance. To ensure versatility and accuracy, it includes annotations like hand dominance (with **7673 instances for the right hand and 1634 for the left**), frame rates, and sign word labels, streamlining it for machine learning applications.

Two distinct versions of video samples were created to cater to different research needs: an original version without frame rate adjustments and another standardized at 30 FPS. Data preprocessing utilized **Mediapipe Holistic**, effectively capturing **543 landmarks per frame**, with a thoughtful approach to managing missing data by substituting zeros. This robust preparation significantly enhances the dataset's usability for deep learning models and benchmarking purposes.

Funded partly by the Innovation Fund of the **Information and Communication Technology Division (ICTD)**, **Government of Bangladesh (2023-2024)**, BdSLW60 not only bridges a crucial gap in sign language recognition research but also positions itself as a valuable resource for developing inclusive technologies in Bangladesh.

Do you think both Convolution Neural Network (CNN) and Recurrent Neural Network (RNN) are required for this sign word recognition task? Justify your answer.

I do think we need **both CNN and RNN** for this task because sign language recognition isn't just about recognizing hand shapes in a single frame - it also involves understanding how the hands move over time.

Why CNN? (Understanding Individual Frames)

- Each video frame contains important spatial information - things like hand shape, finger positioning, and orientation.
- CNNs are great at extracting high-level patterns from images, such as edges and textures, which helps differentiate between different hand positions.
- If we just used raw pixel values, it would be too much noise. **CNN helps reduce complexity by creating compact feature representations.**

Why RNN? (Understanding Motion Over Time)

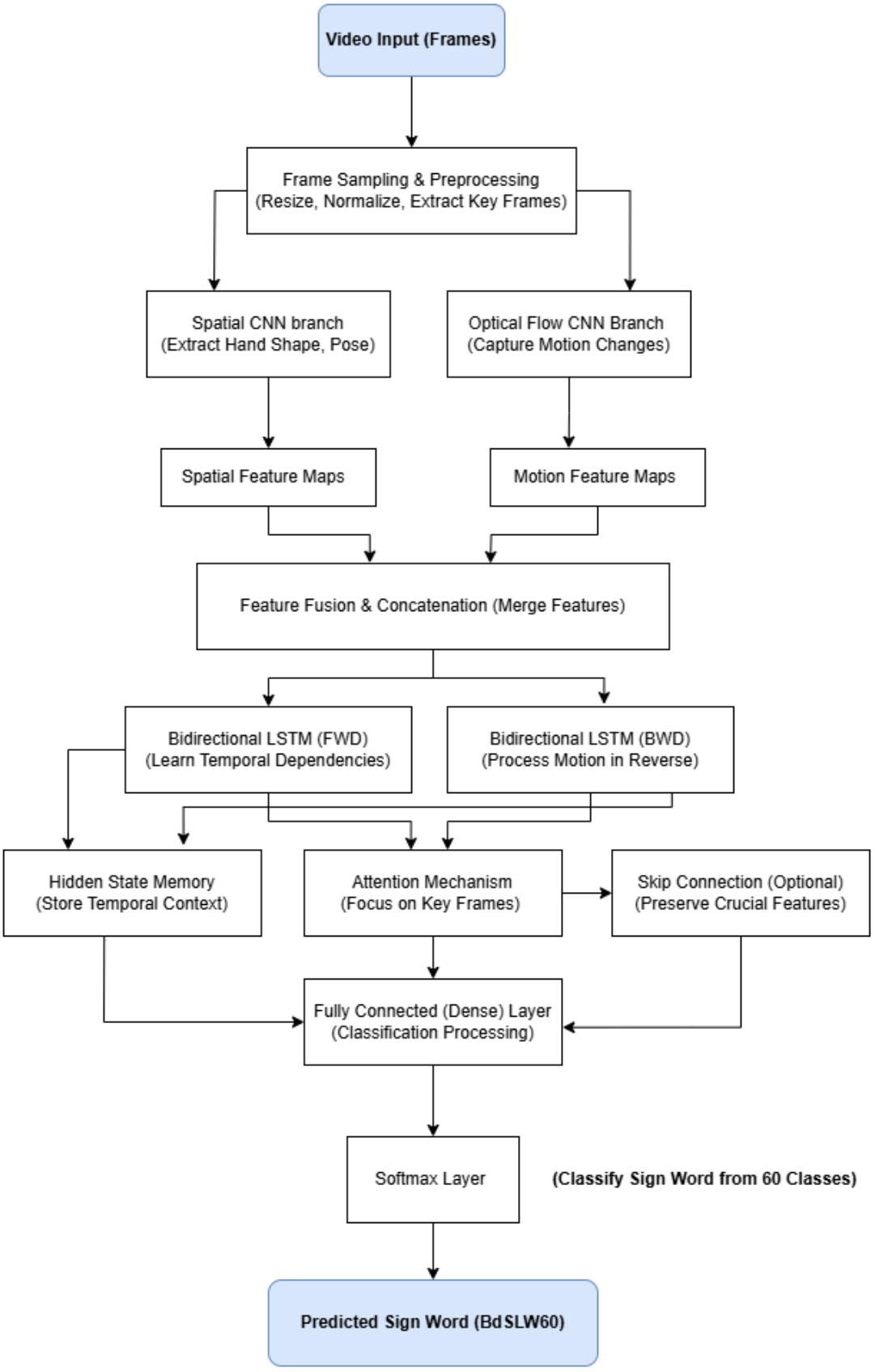
- A single frame won't tell us everything. For example, if someone moves their hand from left to right, we need to track that movement across multiple frames to understand the gesture.
- RNNs are designed to handle sequential data, making them ideal for modeling how hand positions change over time.
- This allows the model to learn the temporal flow of signs, which is crucial because some signs may look similar in a single frame but have completely different meanings based on movement.

Why not just CNN or RNN alone?

- If we only used CNNs, we would lose the motion information and only recognize hand shapes, but not movements.
- If we only used RNNs, they would struggle with processing raw pixel data, which is too complex and inefficient.

So, in my opinion, the best solution is a **CNN+RNN hybrid model -** CNN extracts spatial features from each frame. RNN processes these features over time to learn motion patterns. A fully connected layer classifies the sign word.

Propose a suitable deep learning-based architecture by drawing the block diagram of your proposed solution.



Explain how video frames and sign movement information can be processed through your model architecture.

Step 1: The Model Watches the Video (Frame Sampling & Preprocessing)

First the model needs to convert the video into frames so it can analyze each moment. But, it shouldn't be overfed with too much repetitive information. So, we sample key frames (e.g., every 3rd or 5th frame) to keep only the most important movements. Then resize them to a standard size (e.g., 224×224 pixels) to fit into the model. Then normalize pixel values to prevent brightness differences from messing things up.At last we have a clean, structured sequence of images that represent the sign gesture.

Step 2: The Model Looks at Each Frame Carefully (Feature Extraction with CNNs)

Now that we have individual frames, we want the model to understand what's in each one. This is where two parallel CNN branches come into play:

- **Branch 1: Spatial CNN – "What Do the Hands Look Like?"** - This CNN scans each frame to detect hand shape, finger positioning, and orientation

- **Branch 2: Optical Flow CNN – "How Are the Hands Moving?"** - This CNN is motion-sensitive—it detects how the hand shifts from one frame to another.

Once both CNNs are done, we'll get two feature maps - one describing hand shape (static details) and another describing hand motion (dynamic details).

Step 3: The Model Learns the Hand Movements Over Time (Bi-LSTM)

Now that we have a sequence of features, we need to understand how they change over time.
This is where Bidirectional LSTM (Bi-LSTM) comes in. It's like the memory of the model that keeps track of what happened before and what's happening now.
Instead of just looking forward in time, Bi-LSTM looks both forward and backward, meaning it can recognize movement patterns more accurately.

Why does this matter?
Suppose two signs start the same way but end differently (like "টিভি" vs. "এসি"). If the model only looks forward, it might misinterpret the sign early on. But with Bi-LSTM reading both forward and backward, it can consider the full motion sequence before deciding what sign was made.

Step 4: The Model Pays Attention to the Most Important Frames (Attention Mechanism)

Not every single frame in the video is important. If someone waves really fast, only a few frames will actually contain the key motion details. The Attention Mechanism helps the model prioritize the most relevant frames and ignore the extra noise.

Step 5: The Model Remembers Previous Movements (Hidden State Memory)

Now, what if a sign requires multiple gestures in sequence? This is where Hidden State Memory plays a role. It stores information from earlier frames so the model doesn't forget previous movements when predicting the final sign. The Hidden State Memory remembers early frames and connects them with later ones for more accurate predictions.

Step 6: The Model Sends the Final Processed Information to the Classifier

Now that we have all this clean, motion-aware, memory-enhanced information, the model is finally ready to make a decision. It passes everything through a fully connected (Dense) layer that refines the features. Then, the Softmax layer assigns a probability to each of the 60 sign words in the dataset. The model picks the sign word with the highest probability as the final prediction.

Example:
The model might assign probabilities like this:

"টিভি" → 85%
"এসি" → 10%
"আয়না" → 5%
Since "টিভি" has the highest probability, that's what the model predicts.

By the time the model finishes all these steps, it "knows" what sign word was performed—just like how we understand sign language by watching hand gestures.

Provide relevant mathematical formulations for CNN/RNN operations.

How CNN Extracts Features from a Frame?

Instead of looking at individual pixels (which is messy), CNNs extract meaningful information that can be used later.

Each CNN layer applies convolution, which is basically a weighted sum of pixel values in a small area. Mathematically, it looks like this:

$$X_{i,j,k}^{(l+1)} = \sum_{m} \sum_{n} \sum_{c} W_{m,n,c,k}^{(l)} \cdot X_{i+m,j+n,c}^{(l)} + b_k$$

- $X^{(l)}$ = The image (or feature map) at layer l.
- $W^{(l)}$ = The filter (also called a kernel), which detects patterns.
- b_k = A bias term to adjust the values.

Basically, the filter slides over the image, applies its weights, and generates a new, smaller image that highlights important features.

After convolution, we usually apply:

- **ReLU activation** (to remove negative values).
- **Pooling (Max Pooling or Average Pooling)** to downsample the image while keeping the important features.

How RNN (LSTM) Handles Sequences?

Unlike CNNs, RNNs don't just look at one frame—they take in a series of frames and try to learn how they change over time.

The key part of an LSTM is the memory cell. It has different "gates" that control what information is kept, updated, or discarded.

- Forget Gate (f_t) = Decides what information should be thrown away.

$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$$

- Input Gate (i_t) = Decides what new information should be added.

$$i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$$

- Cell State Update (C_t) = Combines old and new info to update memory.

$$C_t = f_t C_{t-1} + i_t \grave{C}_t$$

- Output Gate (o_t) = Decides what should be passed to the next timestep.

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

- Hidden State (h_t) = The final processed output for that timestep.

$$h_t = o_t \tanh(C_t)$$

- The forget gate clears out useless info (like background noise in frames).
- The input gate updates memory with important new details.
- The output gate decides what to pass to the next step.

Describe how you are going to evaluate your model's performance. Justify your choice on evaluation metric

Once our model is trained, we need to make sure it's actually good at recognizing Bangla Sign Language words. We need concrete numbers to prove it.

Since we're dealing with 60 different words, accuracy alone won't cut it. We need a few different evaluation metrics to see where the model shines and where it struggles.

1. **Accuracy** is the easiest way to check overall performance. It's simply the percentage of correct predictions:

$$Accuracy = Total\ Predictions / Correct\ Predictions$$

But accuracy doesn't tell us where the mistakes are happening. If the model gets "মা" right every time but constantly messes up "দাদা", accuracy alone won't help us fix that issue.

2. **Precision:** Imagine the model predicts "বালু" 100 times. Out of those, it was actually correct 80 times. The rest were incorrect guesses where it should have predicted something else, like "বালতি" or "বাত".

$$Precision = True\ Positives / (False\ Positives + True\ Positives)$$

A high precision means when the model predicts a certain word, it's usually right.

3. **Recall:** Now, let's say in reality, the dataset contained 120 instances of "বালু", but the model only predicted 80 of them correctly.

$$Recall = True\ Positives / (True\ Positives + False\ Negatives)$$

A high recall means the model doesn't miss too many correct words.

4. **F1-Score:** Sometimes, precision is high but recall is low, or vice versa. The F1-score balances both.

$$F1 = 2 * ((Precision * Recall) / (Precision + Recall))$$

This is a more balanced metric that gives us an idea of overall model performance.

5. **Confusion Matrix:** This is super useful because it shows us which signs are getting mixed up.
6. **Top-K Accuracy:** This is useful because sometimes a mistake isn't completely wrong. In sign language, some words are visually similar, and even humans might need extra context to tell them apart.

For example, if we use Top-3 Accuracy, it means:

- The model doesn't have to be 100% right in the first prediction.
- If the correct answer is at least in the top 3 guesses, it still counts as a success.

This is helpful because many signs have small variations that make them look alike, especially in video-based recognition.

Why do these metrics matter?

- Accuracy isn't enough—it doesn't tell us which words are being confused.
- Precision, recall, and F1-score help us see if the model is biased toward common words.
- Confusion matrix tells us where the biggest mistakes happen, so we can fix them.
- Top-k accuracy is useful for real-world scenarios where similar signs exist.

REFERENCES:

1. draw.io - For drawing the diagram
2. overleaf.com - Writing the equation
3. https://www.kaggle.com/datasets/hasaniut/bdslw60 - The provided dataset and relevant details
4. A gentle introduction to LSTM